

20 Python String Practice Problems

Created for Career Credentials by Dr. Amar Panchal
Technical Trainer, Co-founder, & Placement Expert

Basic Manipulation & Analysis

These problems cover the fundamentals of string inspection and basic changes. You'll practice counting characters, reversing strings, and identifying patterns, which are essential building blocks for more complex tasks.

1. Count Vowels

Problem Definition:

Write a function to count the number of vowels (a, e, i, o, u) in a given string. The check should be case insensitive.

Sample Input:

`"Career Credentials"`

Sample Output:

`7`

2. Reverse String

Problem Definition:

Write a function that takes a string and returns its reverse. You must use a loop or string indexing/slicing (implementing your own logic), not built-in reversal methods like `'reversed()'`.

Sample Input:

`"Placement"`

Sample Output:

`"tnemecalP"`

3. Check Palindrome

Problem Definition:

Write a function to check if a string is a palindrome (reads the same forwards and backward). Ignore the case of the letters.

Sample Input:

`"Racacar"`

Sample Output:

`True`

4. Word Counter

Problem Definition:

Write a function that counts the number of words in a string. Assume words are separated by one or more spaces.

Sample Input:

`"Train to get placed"`

Sample Output:

`4`

5. Remove Spaces

Problem Definition:

Write a function to remove all whitespace characters (spaces, tabs, newlines) from a string.

Sample Input:

`"a m a r , c a r e e r"`

Sample Output:

`"amarcareer"`

Case & Character Transformation

This section focusses on altering characters within a string. You will work on changing case, shifting characters based on rules, and managing character frequency, all common tasks in data cleaning and encoding.

6. Title Case

Problem Definition:

Write a function that converts the first letter of every word to uppercase and the rest of the letters in the word to lowercase. (Do not use the built-in `'title()'` method).

Sample Input:

`"this is an example"`

Sample Output:

`"This Is An Example"`

7. Toggle Case

Problem Definition:

Write a function that iterates through a string and converts all uppercase letters to lowercase and all lowercase letters to uppercase.

Sample Input:

`"CoDiNg SKiLiS"`

Sample Output:

`"cOdInG sKiLs"`

8. Caesar Cipher (Simple)

Problem Definition:

Implement a basic Caesar cipher where every letter in a string is shifted forward by a specific number (e.g., 3 places). Only apply the shift to letters (a-z, A-Z); numbers and symbols should remain unchanged.

Sample Input:

`String: "abc", Shift: 3`

Sample Output:

`"def"`

9. Remove Duplicates (Preserve Order)

Problem Definition:

Remove all duplicate characters from a string, ensuring you keep the first occurrence of each character and maintain the original order.

Sample Input:

`"programming"`

Sample Output:

`"programin"`

10. Count Character Frequency

Problem Definition:

Write a function that returns a dictionary where keys are the characters in the string and values are their respective counts.

Sample Input:

`"apple"`

Sample Output:

`{ 'a': 1, 'p': 2, 'l': 1, 'e': 1 }`

Substrings & Indexing

Here, you'll dive into working with parts of strings. These challenges involve finding, extracting, and comparing substrings, which are critical skills for text processing and data parsing.

11. Find First Substring Index

Problem Definition:

Find the starting index of the first occurrence of a smaller substring within a larger string. If the substring is not found, return -1. (You must implement the search logic; do not use the built-in `'find()'` or `'index()'` methods).

Sample Input:

`String: "abracadabra", Substring: "cad"`

Sample Output:

`4`

12. Extract Middle Character(s)

Problem Definition:

Return the middle character of a string. If the string's length is even, return the middle two characters. Sample

Input:

`Input 1: "abcde"`
`Input 2: "abcd"`

Sample Output:

`Output 1: "c"`
`Output 2: "bc"`

13. Remove Nth Index

Problem Definition:

Write a function to remove the character at the Nth index from a string (where N is 0-indexed).

Sample Input:

`String: "Python", N: 2`

Sample Output:

`"Pyhon"`

14. Check Anagrams

Problem Definition:

Determine if two strings are anagrams of each other (contain the same characters with the same frequency). Your check should ignore case and spaces.

Sample Input:

`Sr1: "listen", Sr2: "silent"`

Sample Output:

`True`

15. Longest Word

Problem Definition:

Find the length of the longest word in a sentence. Assume words are separated by a single space.

Sample Input:

`"The technical trainer"`

Sample Output:

`9`

Advanced Logic

These problems require more complex logic and multi-step thinking. You'll tackle challenges like string compression, custom validation, and simple expression evaluation, pushing your problem-solving skills.

16. Check for Only Digits

Problem Definition:

Write a function to check if a string contains only digits (0-9). You must implement this using a loop and conditional checks, do not use the built-in string method `'isdigit()'`.

Sample Input:

`Input 1: "12345"`
`Input 2: "123a45"`

Sample Output:

`Output 1: True`
`Output 2: False`

17. String Compression

Problem Definition:

Implement a basic string compression method that replaces repeated characters with the character followed by its count. For example, `"aaabbb"` becomes `"a3b3c1"`. Only return the compressed string if it is shorter than the original string; otherwise, return the original string.

Sample Input:

`Input 1: "aaabbbcccddde"`
`Input 2: "abc"`

Sample Output:

`Output 1: "a3b3c4d2e1"`
`Output 2: "abc"`

18. Check Prefix/Suffix

Problem Definition:

Determine if a string starts with a specific prefix AND ends with a specific suffix. You must implement the check logic; do not use the built-in `'startswith()'` or `'endswith()'` methods.

Sample Input:

`String: "GeminIAI", Prefix: "Gem", Suffix: "AI"`

Sample Output:

`True`

19. Interleave Strings

Problem Definition:

Combine two strings by alternating characters from each. Start with the first string. If one string is longer, append the remaining characters from the longer string to the result.

Sample Input:

`Sr1: "abc", Sr2: "xyz123"`

Sample Output:

`"axbycz123"`

20. Simple String Calculator

Problem Definition:

Evaluate a string that contains a very simple arithmetic expression (e.g., `"10+5"`, `"20-8"`) and return the integer result. Assume the expression only contains two positive integers and a single operator, which is either addition (+) or subtraction (-).

Sample Input:

`"100-25"`

Sample Output:

`75`

